

로그 RCA 를 위한 증거 보존형 KV Cache 압축 기법

LLM 을 활용한 로그 기반 RCA 의 필요성과 실무적 가치, 그리고 로그 특화 KV cache 압축의 이유

프로젝트 한 문장

본 프로젝트는 긴 로그를 LLM 으로 RCA 할 때, 로그의 희귀도·위치·레벨 같은 고유 신호를 활용해 원인 증거를 보존하면서 KV cache 를 압축하는 로그 특화 압축기를 제안한다.

1. 서론

현대의 대규모 서비스는 수많은 서버, 데이터베이스, API, 캐시, 네트워크, 마이크로서비스가 복잡하게 연결되어 운영된다. 이러한 환경에서 장애가 발생했을 때 중요한 것은 단순히 문제가 발생했다는 사실을 아는 것이 아니라, 원인이 무엇인지 파악하고 어떤 조치를 해야 하는지 빠르게 결정하는 것이다.

로그는 장애 원인을 찾는 데 중요한 단서를 제공한다. 그러나 실제 운영 환경에서 생성되는 로그는 매우 방대하고, 대부분은 정상 동작이나 반복적인 시스템 메시지에 해당한다. 이 때문에 장애의 핵심 원인은 수많은 로그 속에 묻히기 쉽다.

본 보고서는 로그분석을 선택한 이유, 그중 RCA 를 선택한 이유, 실제 실무에서 나타나는 어려움, LLM 을 활용한 로그분석의 필요성, 이를 통해 기대할 수 있는 실무적 이점, 그리고 프로젝트의 핵심 방향인 로그 특화 KV cache 압축을 함께 보아야 하는 이유를 정리한다.

2. 왜 하필 로그분석인가?

LLM 을 활용할 수 있는 분야는 일반 문서 QA, 챗봇, 요약, 코드 생성, 검색 보조 등 다양하다. 그중 로그분석을 선택한 이유는 로그가 LLM 기반 분석 연구에 적합한 특성을 갖고 있기 때문이다.

2.1 평가 기준을 세우기 쉽다

일반 챗봇 응답, 창의적 글쓰기, 요약은 정답이 여러 개일 수 있고 평가가 주관적이다. 반면 로그분석은 장애가 실제로 발생했는지, 어떤 서비스가 원인이었는지, 어떤 라인이 이상이었는지 사후 조사나 라벨을 통해 확인할 수 있다. 공개 로그 데이터셋과 Top-K 정확도, F1, precision, recall 같은 지표를 활용할 수 있다는 점도 장점이다.

2.2 산업적 임팩트를 수치화하기 쉽다

로그분석은 산업적 효과를 수치로 설명하기 쉽다. 장애 대응 시간은 MTTR로 측정할 수 있고, 다운타임 비용, SRE 인력 비용, 알람 처리량, LLM 호출 비용도 정량화할 수 있다. 즉 “좋아 보인다”가 아니라 비용, 시간, 처리량 같은 지표로 실무 가치를 설명할 수 있다.

2.3 주변 인프라는 이미 성숙해 있다

로그를 수집하고 저장하고 검색하고 시각화하는 도구는 이미 오랫동안 발전해왔다. OpenTelemetry, Fluentd, Elasticsearch, Loki, Grafana, Kibana, Datadog, Splunk 같은 도구들이 대표적이다. 즉 수집, 저장, 검색, 시각화 층은 성숙했지만, 로그를 해석하고 원인을 판단하는 분석 층은 아직 미성숙하다.

정리하면, 로그분석은 평가 가능성, 산업적 수요, 성숙한 주변 인프라 위의 미해결 분석 문제라는 세 조건이 동시에 맞는 도메인이다.

3. 로그분석 문제 중 RCA 를 선택한 이유

로그분석에는 이상 탐지, 로그 분류, 로그 요약, 장애 예측, 원인 분석 등 여러 과제가 존재한다. 이 중 RCA 를 선택한 이유는 RCA 가 실제 장애 대응 과정에서 가장 직접적으로 실무 가치와 연결되기 때문이다.

이상 탐지는 시스템에 문제가 발생했다는 사실을 알려주는 역할을 한다. 로그 요약은 발생한 사건을 간단히 정리해주는 역할을 한다. 그러나 SRE 나 DevOps 엔지니어가 실제 장애 상황에서 가장 알고 싶은 것은 단순히 문제가 있다는 사실이 아니라, 왜 문제가 발생했는가이다.

장애 상황에서 중요한 질문은 다음과 같다.

- 어떤 서비스에서 문제가 시작되었는가?
- 어떤 로그가 원인을 보여주는가?
- 현재 보이는 ERROR 로그는 원인인가, 아니면 원인의 결과인가?
- 어떤 조치를 해야 장애를 복구할 수 있는가?

이 질문에 답하는 과정이 RCA 이다. RCA 는 단순히 로그를 읽는 작업이 아니라, 장애의 원인을 찾아 실제 조치로 이어지는 핵심 단계이다. 따라서 로그분석의 여러 과제 중 RCA 는 SRE 의 업무와 MTTR, 즉 평균 복구 시간 단축에 가장 직접적으로 연결된다.

따라서 RCA를 선택한 이유는, 단순 탐지나 요약보다 실무적 가치가 크고 장애 대응의 핵심 질문인 “왜 문제가 발생했는가”에 직접 답하기 때문이다.

4. 실무에서 겪는 로그분석의 어려움

실무에서 SRE와 DevOps 팀이 겪는 로그분석의 어려움은 크게 네 가지로 정리할 수 있다.

4.1 로그의 양이 너무 많다

장애가 발생하면 하나의 서비스 로그만 확인하는 것이 아니다. API 서버, 데이터베이스, 캐시, 인증 서비스, 네트워크, 배포 로그 등 여러 계층의 로그를 함께 확인해야 한다. 이 과정에서 수만 줄에서 수십만 줄의 로그가 생성될 수 있다.

그러나 이 중 대부분은 정상 요청, health check, 반복적인 INFO 로그, 또는 원인의 결과로 발생한 증상 로그이다. 실제 원인은 이 거대한 로그 더미 속의 일부 라인에 숨어 있다.

4.2 메트릭은 증상을 보여주지만 원인은 로그에서 확인해야 한다

대시보드나 메트릭은 에러율 증가, latency 증가, CPU 사용량 증가, 요청 실패율 증가와 같은 증상을 빠르게 보여준다. 그러나 이러한 수치만으로는 왜 문제가 발생했는지 알기 어렵다. 실제 원인을 찾으려면 로그를 통해 어떤 이벤트가 발생했는지, 어떤 서비스에 문제가 시작되었는지, 어떤 메시지가 원인을 설명하는지 확인해야 한다.

즉 증상은 빠르게 보이지만, 원인을 찾고 검증하는 과정은 여전히 사람의 판단에 크게 의존한다.

4.3 LLM의 추론 정확도가 아직 프로덕션 기준에 부족하다

기업들이 로그 인프라 최적화는 활발하게 진행하면서도 LLM 기반 로그분석에는 조심스러운 이유는, LLM의 추론 정확도가 아직 핵심 운영 업무에 바로 적용하기에는 부족하기 때문이다.

제공 자료에서는 Meta의 AI-assisted RCA가 back-testing에서 Top-5 정답 포함률 42% 수준이고, OpenRCA 벤치마크에서 5개 프런티어 모델의 완전 정답률이 3.9~12.5%에 그친다고 정리되어 있다. 또한 Datadog과 Splunk 같은 상용 도구들도 완전 자동 조치보다는 보조적 기능에 머문다고 설명한다.

이는 LLM이 단순히 로그를 읽는 수준의 작업은 어느 정도 수행할 수 있지만, 여러 로그를 연결해 왜 장애가 발생했는가를 추론하는 인과추론에는 아직 약하다는 점을 보여준다.

4.4 로그는 LLM에게 특히 어려운 입력이다

로그는 일반 자연어 문서와 다르다. 로그는 반정형 데이터이며, 타임스탬프, 숫자, UUID, 에러 코드, 서비스명, 식별자, 경로, 포트 번호 등이 섞여 있다. 이런 데이터는 LLM이 학습 과정에서 많이 본 자연어와 다르기 때문에 분포 미스매치가 발생할 수 있다.

또한 로그 분석은 시간적·인과적 추론을 요구한다. 단순히 A와 B가 같이 등장했다는 것이 아니라, A가 먼저 발생했고 그 결과 B가 발생했으며 이것이 C 장애로 이어졌는지를 판단해야 한다. 제공 자료에서도 LLM 로그분석의 어려움으로 토큰화 문제, 시계열 인지, 환각, 분포 미스매치, 멀티시그널 통합, RAG 검색 실패, 실시간성·비용, 데이터 거버넌스 등이 제시되어 있다.

결국 실무에서는 로그는 너무 많고, 원인은 일부 라인에 숨어 있으며, LLM에 그대로 맡기면 비용, 지연, 환각, 잘못된 추론 문제가 생기는 딜레마가 발생한다.

5. 왜 LLM을 통한 로그분석을 활용하는가?

그럼에도 LLM을 로그분석에 활용하려는 이유는 로그가 텍스트와 맥락을 포함한 데이터이기 때문이다.

전통적인 로그 분석 방식은 키워드 검색, ERROR/WARN 필터링, 정규식 기반 탐지, 통계적 이상 탐지에 강점을 가진다. 그러나 RCA에서는 이러한 방식만으로 충분하지 않은 경우가 많다.

예를 들어 ERROR 로그가 많이 발생했다고 해서 그 ERROR가 반드시 원인은 아니다. 여러 서비스에서 나타나는 timeout이나 failed request는 실제 원인이 아니라 앞선 장애로 인해 발생한 결과일 수 있다. 반대로 실제 원인은 겉보기에는 평범한 INFO 로그, 설정 변경 메시지, 배포 직후의 경고, 특정 서비스의 비정상적인 응답 패턴일 수도 있다.

LLM은 이런 상황에서 다음과 같은 역할을 할 수 있다.

- 여러 로그 줄을 문맥적으로 읽기
- 증상과 원인 후보를 자연어로 연결하기
- SRE가 확인할 가설 제시하기
- 의심 로그와 근거를 설명하기
- 신규 인력에게 장애 흐름을 설명하기

따라서 LLM 기반 로그분석의 목표는 LLM이 혼자 장애를 완전히 해결하는 것이 아니다. 현실적인 목표는 SRE가 확인할 만한 원인 후보와 근거 로그를 빠르게 제시하는 것이다.

LLM 로그분석은 장애 원인을 단독으로 확정하는 기술이 아니라, SRE 가 빠르게 검증할 수 있는 원인 후보와 근거 로그를 제시함으로써 RCA 과정을 단축하는 보조 기술이다.

6. 실무에서 기대할 수 있는 이점

앞선 장에서 LLM 기반 로그 RCA 의 실무적 가치를 설명했지만, 실제 현장에서 SRE 가 어떻게 로그를 보고 장애를 분석하는지, 그리고 본 프로젝트가 그 과정의 어느 부분을 돕는지 분명히 할 필요가 있다.

6.1 SRE 가 로그분석을 하는 실제 흐름

장애가 발생하면 SRE 는 먼저 알림과 대시보드를 통해 증상을 확인한다. 예를 들어 에러율 증가, latency 증가, 특정 API 실패, CPU 나 메모리 사용량 증가 같은 신호를 보고 어떤 서비스와 시간대에서 문제가 시작되었는지 좁힌다.

그다음 관련 로그를 검색한다. 이때 SRE 는 시간 범위, 로그 레벨, 배포 시점, 장애 직전·직후의 위치 같은 조건으로 로그를 좁혀가며 원인 후보를 찾는다. 하지만 실제 로그에는 정상 요청, health check, 반복 INFO, 원인의 결과로 발생한 ERROR 가 섞여 있어 원인 라인을 바로 찾기 어렵다.

이후 SRE 는 로그를 바탕으로 원인 가설을 세운다. 예를 들어 “인증 서비스에서 timeout 이 먼저 발생했고, 그 결과 API 서버의 failed request 가 증가했다”처럼 증상과 원인을 구분한다. 마지막으로 로그, 메트릭, 트레이스, 배포 이력, 코드 변경 내용을 함께 확인해 가설을 검증하고, 롤백·재시작·설정 변경·스케일 조정 같은 조치를 결정한다.

정리하면 SRE 의 로그분석 흐름은 다음과 같다.

- 알림·대시보드로 증상 확인
- 문제가 발생한 서비스와 시간대 좁히기
- 관련 로그 검색 및 필터링
- 원인 후보 라인과 증거 로그 찾기
- 원인 가설 수립
- 로그·메트릭·트레이스·배포 이력으로 검증
- 최종 조치 결정

6.2 본 프로젝트가 도와주는 부분

본 프로젝트는 SRE 의 모든 판단을 자동으로 대체하려는 것이 아니다. 핵심은 SRE 가 로그를 검토하는 과정에서 가장 시간이 많이 드는 부분, 즉 긴 로그 속에서 원인 후보 라인과 근거 로그를 찾는 단계를 줄이는 것이다.

SRE 가 로그를 볼 때 어려운 부분은 “무슨 일이 일어났는가”보다 “어떤 로그가 진짜 원인을 보여주는가”이다. 따라서 본 프로젝트는 긴 로그를 LLM 으로 RCA 할 때, 반복적이고 정보량이 낮은 로그의 KV 는 줄이고, 희귀한 원인 후보 라인과 중간에 묻힌 증거 라인의 KV 는 보존하도록 돕는다.

즉 본 프로젝트가 돕는 지점은 다음과 같다.

- SRE 가 모든 로그를 처음부터 읽지 않아도 되도록 핵심 후보를 남긴다.
- 반복 INFO, health check, 중복 에러 같은 노이즈의 비중을 줄인다.
- 설정 변경, 드문 WARN, 장애 직전 희귀 템플릿 같은 원인 후보를 보존한다.
- 긴 로그 중간에 묻힌 증거가 압축 과정에서 사라지지 않도록 한다.
- LLM 이 원인 후보와 근거를 제시하면 SRE 가 이를 검증하고 최종 판단한다.

따라서 본 프로젝트의 역할은 자동 복구나 자동 조치가 아니라, SRE 가 RCA 를 수행할 때 확인해야 할 원인 증거를 덜 잃게 만드는 것이다. 이 점에서 로그 특화 KV cache 압축기는 운영 효율을 높이는 동시에, SRE 의 원인 검증 과정을 보조하는 기술이다.

6.3 SRE 가 로그를 처음부터 모두 확인하지 않아도 된다

기존에는 SRE 가 수많은 로그 속에서 직접 원인 후보를 찾아야 했다. 하지만 LLM 이 관련 로그를 요약하고, 의심되는 원인 후보와 근거 라인을 제시한다면 SRE 는 모든 로그를 처음부터 읽는 대신 후보 중심으로 검증을 시작할 수 있다. 이는 로그 탐색 시간을 줄이고, 장애 대응 초기 판단 속도를 높이는 데 도움이 된다.

6.4 원인 후보에 도달하는 시간이 줄어든다

장애 대응에서 중요한 것은 가능한 한 빠르게 원인 후보를 좁히는 것이다. LLM 은 여러 로그를 문맥적으로 읽고, 서로 관련 있어 보이는 이벤트를 연결해 SRE 가 확인할 만한 가설을 제시할 수 있다. 이를 통해 SRE 는 무작위로 로그를 탐색하는 대신, 우선순위가 높은 후보부터 확인할 수 있다.

6.5 MTTR 단축 가능성이 생긴다

MTTR 은 장애가 발생한 뒤 복구까지 걸리는 시간을 의미한다. RCA 과정이 빨라지면 원인 파악과 조치 결정이 빨라지고, 결과적으로 복구 시간 단축으로 이어질 수 있다. 따라서 LLM 기반 RCA 는 단순한 편의 기능이 아니라 장애 대응 비용을 줄일 가능성이 있는 실무적 기술이다.

6.6 신규 SRE 나 시스템 이해도가 낮은 팀원도 장애 흐름을 이해하기 쉬워진다

숙련된 SRE 는 로그를 보고 원인과 증상을 비교적 빠르게 구분하지만, 신규 인력은 어떤 로그가 중요한지 판단하기 어렵다. LLM 이 의심 서비스, 근거 로그, 원인 후보, 다음 확인 사항을 설명해준다면 신규 인력도 장애의 흐름을 이해하고 대응 과정에 참여하기 쉬워진다. 이는 팀 내 대응 품질 편차를 줄이고, 온보딩 비용을 낮추는 데 도움이 될 수 있다.

6.7 완전 자동화가 아니라 검증 가능한 보조 도구가 된다

실무에서는 LLM 이 자동으로 조치까지 수행하는 방식은 위험할 수 있다. 환각이나 잘못된 인과추론이 곧 잘못된 조치로 이어질 수 있기 때문이다. 따라서 현실적인 활용 방식은 LLM 이 원인 후보와 근거를 제시하고, SRE 가 이를 확인한 뒤 최종 판단하는 구조이다.

정리하면, LLM 기반 로그 RCA는 로그 탐색 시간 감소, 원인 후보 도달 시간 단축, 신규 인력의 장애 이해 보조, MTTR 단축 가능성이라는 실무적 가치를 제공한다.

7. 왜 KV Cache 까지 봐야 하는가?

앞선 내용에서 LLM 기반 로그 RCA의 필요성은 “SRE가 긴 로그 속에서 원인 후보와 근거를 빠르게 찾도록 돕는 것”으로 정리할 수 있다. 그러나 이 목표를 실제 시스템으로 구현하려면 단순히 LLM이 로그를 읽게 하는 것만으로는 부족하다. 긴 로그를 LLM에 입력하는 순간, 비용·지연·정확도 문제뿐 아니라 추론 과정의 메모리 병목도 함께 발생하기 때문이다.

KV cache에 주목하는 이유는 두 축으로 정리할 수 있다. 첫째는 운영 효율성이다. 긴 로그를 LLM에 넣으면 context length가 길어지고, 그에 따라 KV cache가 커져 GPU 메모리, 지연시간, 처리량, 운영 비용의 병목이 된다. 둘째는 분석 품질 보존이다. KV cache를 무작정 줄이면 RCA에 필요한 원인 라인과 증거 라인의 KV까지 삭제될 수 있고, 그러면 메모리는 줄었지만 원인 분석 품질은 망가질 수 있다.

7.1 첫 번째 축: 운영 효율성

LLM은 긴 입력을 처리할 때 이전 토큰의 Key와 Value를 KV cache로 저장한다. 입력 로그가 길어질수록 context length가 커지고, context length가 커질수록 KV cache도 커진다. 제공 자료에서도 KV cache 크기는 context length와 batch size에 선형적으로 증가하며, 긴 sequence와 큰 batch에서는 KV cache memory가 main bottleneck이 된다고 정리되어 있다.

즉 문제 흐름은 다음과 같다.

- 장애 로그가 길다.
- LLM에 넣는 토큰 수가 많아진다.
- context length가 길어진다.
- KV cache가 커진다.
- GPU 메모리 사용량이 증가한다.
- 동시에 처리할 수 있는 RCA 요청 수가 줄어든다.
- 응답 지연과 운영 비용이 증가한다.

따라서 KV cache를 보는 첫 번째 이유는 LLM 기반 로그 RCA를 실제 운영 환경에서 더 빠르고 효율적으로 돌리기 위해서다. 같은 GPU로 더 긴 로그를 처리하고, 더 많은 RCA 요청을 동시에 처리하며, 응답 지연과 운영 비용을 줄이려면 KV cache 최적화가 필요하다.

7.2 두 번째 축: 로그분석 품질 보존 또는 개선

하지만 본 프로젝트의 더 중요한 연구 포인트는 단순 운영비용 절감이 아니다. 단순히 KV cache를 줄이면 중요한 원인 라인의 KV까지 버릴 수 있다. 그러면 메모리는 줄었지만, RCA는 망가질 수 있다.

따라서 이 프로젝트는 아무 KV나 버리는 것이 아니라, 로그 고유 신호를 이용해서 원인일 가능성이 높은 라인은 남기고, 반복적이고 정보량 낮은 라인은 버리는 방향을 잡는다. 이렇게 하면 압축된 상태에서도 로그분석 품질을 지키거나, 경우에 따라 더 낮게 만들 수 있는 가능성이 생긴다.

7.3 Gleaner를 어떻게 참고하는가?

Gleaner는 본 프로젝트의 방법론을 그대로 제공하는 연구는 아니다. Gleaner는 모델에 입력하기 전 로그나 트레이스를 샘플링하는 방식에 가깝고, 본 프로젝트는 LLM 내부에서 생성된 KV cache 중 무엇을 남길지 결정하는 문제를 다룬다. 따라서 Gleaner를 “그대로 차용한 방법”으로 설명하면 입력 샘플링과 KV cache 압축이 혼동될 수 있다.

그럼에도 Gleaner는 중요한 문제의식을 제공한다. Gleaner는 RCA에서 모든 데이터를 그대로 사용하는 것이 항상 최선은 아니며, 적절한 샘플링을 통해 노이즈를 줄이면 RCA 성능이 오히려 좋아질 수 있음을 보여준다. 즉 압축은 단순히 정보를 잃는 행위가 아니라, 반복적이고 정상적인 노이즈를 줄여 원인 신호를 더 두드러지게 만드는 과정이 될 수도 있다.

본 프로젝트는 이 문제의식을 KV cache 압축으로 옮겨 검증한다. Gleaner가 입력 샘플링 단계에서 “축소가 아니라 증폭” 가능성을 보였다면, 우리는 LLM 내부 KV cache 압축에서도 로그의 회귀도, 위치, 레벨 같은 고유 신호를 이용해 회귀 원인 라인과 중간 증거

를 더 잘 보존할 수 있는지 확인한다. 따라서 Gleaner는 방법론의 직접 차용이 아니라, “줄이는 것이 RCA에 도움이 될 수 있다”는 주장을 정당화하는 선행 근거로 사용된다.

핵심은 “KV cache를 줄인다”가 아니라, “KV cache를 줄이면서도 RCA에 필요한 증거를 덜 잃게 만든다”는 것이다.

7.4 입력 필터링만으로 충분하지 않은 이유

로그를 줄이는 가장 쉬운 방법은 LLM에 넣기 전에 미리 필터링하는 것이다. 예를 들어 ERROR/WARN 로그만 남기거나, 특정 시간대의 로그만 자르거나, 키워드 기반으로 관련 로그를 뽑는 방식이 있다. 실제로 LogSage도 Key Log Filtering, Key Log Expansion, Token Overflow Pruning 같은 전처리 과정을 통해 토큰 소비를 크게 줄인다.

하지만 전처리 필터링만으로는 한계가 있다. 너무 공격적으로 필터링하면 ERROR나 FAILED 같은 키워드가 없는 중요한 맥락 정보가 빠질 수 있고, 그 결과 모델이 추측을 하거나 오진할 위험이 커진다.

로그 RCA에서 실제 원인은 항상 눈에 띄는 ERROR 라인으로 나타나지 않는다. 오히려 원인 라인은 다음과 같은 형태일 수 있다.

- 평범해 보이는 INFO 로그
- 드물게 등장하는 설정 변경 로그
- 장애 직전의 희귀 템플릿
- 중간 위치에 묻힌 timeout 전조
- 장애 직전·직후에 나타난 희귀한 전조 로그

따라서 단순 필터링은 눈에 띄는 로그는 잘 남길 수 있지만, 희귀한 원인 라인이나 긴 로그 중간에 묻힌 증거를 놓칠 수 있다. 이 지점에서 KV cache 압축을 로그 특화로 설계할 필요가 생긴다.

7.5 왜 희귀한 원인 라인을 보존해야 하는가?

로그에서는 대부분의 줄이 정상 로그다. 반복 INFO, health check, 정상 요청 로그, 반복되는 timeout 결과, 같은 에러의 대량 반복이 대부분을 차지한다. 반대로 진짜 원인은 자주 등장하는 로그가 아니라, 드물게 나타나는 특이한 라인일 수 있다.

예를 들어 설정 변경, 배포 직후 경고, 평소와 다른 리소스 메시지, 특정 서비스의 드문 WARN, 평범해 보이는 INFO지만 장애 직전 한 번만 등장한 로그가 실제 원인 후보일 수 있다. 이런 라인은 단순 attention 기준이나 최근성 기준으로는 버려질 수 있다.

따라서 우리 프로젝트는 로그의 희귀도, 레벨, 시간, 템플릿, 위치 같은 신호를 이용해 희귀하지만 중요한 원인 후보 라인을 남기려 한다. 이것은 단순 비용 절감이 아니라 RCA 증거 보존에 해당한다.

7.6 왜 중간에 묻힌 증거를 보존해야 하는가?

긴 로그에서 원인이 항상 맨 앞이나 맨 뒤에 있는 것은 아니다. 장애 로그는 보통 시간순으로 길게 쌓이고, 중요한 원인 라인이 중간에 있을 수 있다.

그런데 LLM은 긴 context에서 중간 정보를 잘 놓치는 lost-in-the-middle 문제를 겪을 수 있다. 만약 일반 압축법이 최근 로그만 남기거나 attention이 강한 일부 토큰만 남긴다면, 긴 로그 중간에 묻힌 원인 증거가 사라질 수 있다.

그래서 본 프로젝트는 로그 중간에 묻혀 있지만 원인 후보일 가능성이 있는 라인을 보존하려 한다. 이 역시 운영비 절감이라기보다 분석 품질 유지 또는 개선과 관련된 부분이다.

7.7 KV cache 압축은 로그분석을 더 잘하게 만드는가?

정확히 말하면, KV cache 압축 자체가 LLM의 추론 능력을 올려주는 것은 아니다. KV cache를 건드린다고 모델이 갑자기 인과추론을 더 잘하게 되는 것은 아니다.

하지만 압축 상황에서는 무엇을 남기고 무엇을 버리느냐가 RCA 품질에 직접 영향을 준다. 같은 양을 압축하더라도 원인 후보 라인을 버리면 RCA가 망가질 수 있고, 반복적이고 정보량 낮은 로그를 버리면 RCA 품질을 유지할 수 있다.

따라서 이 프로젝트의 안전한 주장은 “full log보다 무조건 더 잘한다”가 아니다. 목표는 “같은 압축률에서 일반 압축법보다 원인 증거를 더 잘 보존한다”는 것이다. 증거 보존이 잘 되면 RCA 답변 품질도 유지되거나 개선될 가능성이 있지만, 이는 실험으로 확인해야 하는 주장이다.

7.8 왜 로그 고유 신호를 써야 하는가?

일반적인 KV cache 압축 방법은 보통 attention score, 최근성, 고정 budget 같은 기준을 사용한다. 하지만 로그 RCA에서는 일반 문서와 다른 특징이 있다. 로그에는 레벨, 시간, 위치, 템플릿, 반복성, 희귀성 같은 구조적 신호가 존재한다.

이 신호들은 일반 자연어 문서에는 잘 없는 구조다. 그래서 로그에서는 모델 내부 attention만 보지 않고도, 어떤 줄이 원인 후보일 가능성이 높은지 비교적 싸게 추정할 수 있다. 로그 특화 KV cache 압축은 단순히 최근 로그나 attention이 높은 로그만 남기는 것이 아니라, 희귀하지만 원인일 가능성이 있는 라인과 긴 로그 중간에 묻힌 증거 라인을 보존하는 방향으로 설계되어야 한다.

7.9 이 프로젝트의 핵심 문제 정의

LLM 기반 로그 RCA에서는 긴 로그를 처리해야 하지만, 긴 로그는 비용·지연·환각뿐 아니라 KV cache 메모리 병목을 만든다. 기존의 일반 KV cache 압축법은 최근성이나 attention 중심으로 동작하기 때문에, 로그에서 중요한 회귀 원인 라인이나 중간에 묻힌 증거를 버릴 수 있다. 따라서 본 프로젝트는 로그의 고유 신호인 레벨, 시간, 위치, 템플릿 회귀도, 반복성 등을 활용해 원인 후보 라인의 KV cache를 보존하는 로그 특화 압축기를 만들고, 일반 압축법보다 원인 라인을 더 잘 보존함을 실험으로 검증한다.

본 프로젝트는 긴 로그를 KV cache로 압축할 때, 로그 고유 신호를 이용해 회귀한 원인 라인과 중간에 묻힌 증거를 보존하는 로그 특화 압축기를 제안한다.

7.10 실무적으로 얻는 것

이 최적화가 성공하면 실무에서는 다음 이점이 생긴다.

- 같은 GPU로 더 긴 로그를 처리할 수 있다.
- 동시 RCA 요청 처리량이 늘어난다.
- SRE가 확인해야 할 핵심 증거를 덜 잃는다.
- LLM 로그 RCA를 데모가 아니라 운영 가능한 시스템으로 만들 가능성이 커진다.

특히 세 번째 이점이 이 프로젝트의 핵심이다. 무작정 압축하면 원인 라인이 사라질 수 있지만, 로그 특화 신호를 이용하면 회귀한 원인 후보와 중간 증거를 더 잘 보존할 수 있다. 따라서 KV cache는 로그 RCA가 필요한 이유 자체는 아니지만, LLM 로그 RCA를 실무 규모로 돌릴 때 생기는 핵심 운영 병목이자, RCA 증거 보존을 설계해야 하는 중요한 층위이다.